
Trading Book PnL FIFO

Technical Reference

`automation/task_pnl_trading_book.py`

`risk_models/pnl_fifo.py`

Pipeline: `main_run_pnl_fifo` → `initiate_pnl_engine` → `pnl_realised_polars`

May 7, 2026 | Author: F. Balloni — Risk Modelling

Contents

1	Overview	3
2	Top-Level Task: <code>main_run_pnl_fifo</code>	3
2.1	Purpose	3
2.2	Pipeline Flow	4
2.3	Incremental Date Logic	4
3	<code>initiate_pnl_engine</code> — Day-Loop Orchestrator	5
3.1	Function Signature	5
3.2	Detailed Flow	6
3.3	Cache Carry-Forward	6
3.4	Outputs	7
4	<code>pnl_realised_polars</code> — FIFO Calculation Engine	7
4.1	Role in the Call Stack	7
4.2	Input / Output	7
4.3	Internal Flow	8
4.4	Data Structures	8
4.5	Mathematical Formulation	9
4.5.1	Sign Convention	9
4.5.2	Sell Trade ($q_i < 0$): Long Book Realisation	9
4.5.3	Buy Trade ($q_i > 0$): Short Cover	9
4.5.4	Total Realised P&L per Trade	9
4.5.5	Unrealised P&L	10
4.5.6	P&L Identity	10
5	FIFO Matching Examples	10
5.1	Standard Sell Against Long Book	10
5.2	Sell Exceeds Available Longs — Short Queue Created	10
6	Key Implementation Notes	11
7	Infrastructure: AWS ECR & Airflow Orchestration	11
7.1	Docker Image on AWS ECR	11
7.2	Snowflake Credentials Injection	11
7.3	Airflow DAG	12
7.3.1	DAG Workflow	12
7.3.2	Task Descriptions	12
8	Full Call-Stack Summary	13
A	Source Code: <code>pnl_realised_polars</code>	13

Overview

The pipeline computes **realised and unrealised P&L** for the fractional-share trading book using a strict **First-In, First-Out (FIFO)** matching methodology. The computation runs *incrementally*: it reads the latest cached open-position snapshot from Snowflake, processes all new trades up to yesterday, and uploads refreshed position and cache parquet files to S3.

Accounts processed:

Account	Data Source	Trade Query
9800001301	share_booking	old
9800003301	share_booking	old
9800001601	inquisitor	new
9800003601	inquisitor	new

Top-Level Task: `main_run_pnl_fifo`

Purpose

`main_run_pnl_fifo` is the entry point called by the Airflow DAG. It queries the Snowflake P&L cache table for the last processed date per account, then calls `initiate_pnl_engine` for each account with the appropriate incremental date window.

Pipeline Flow

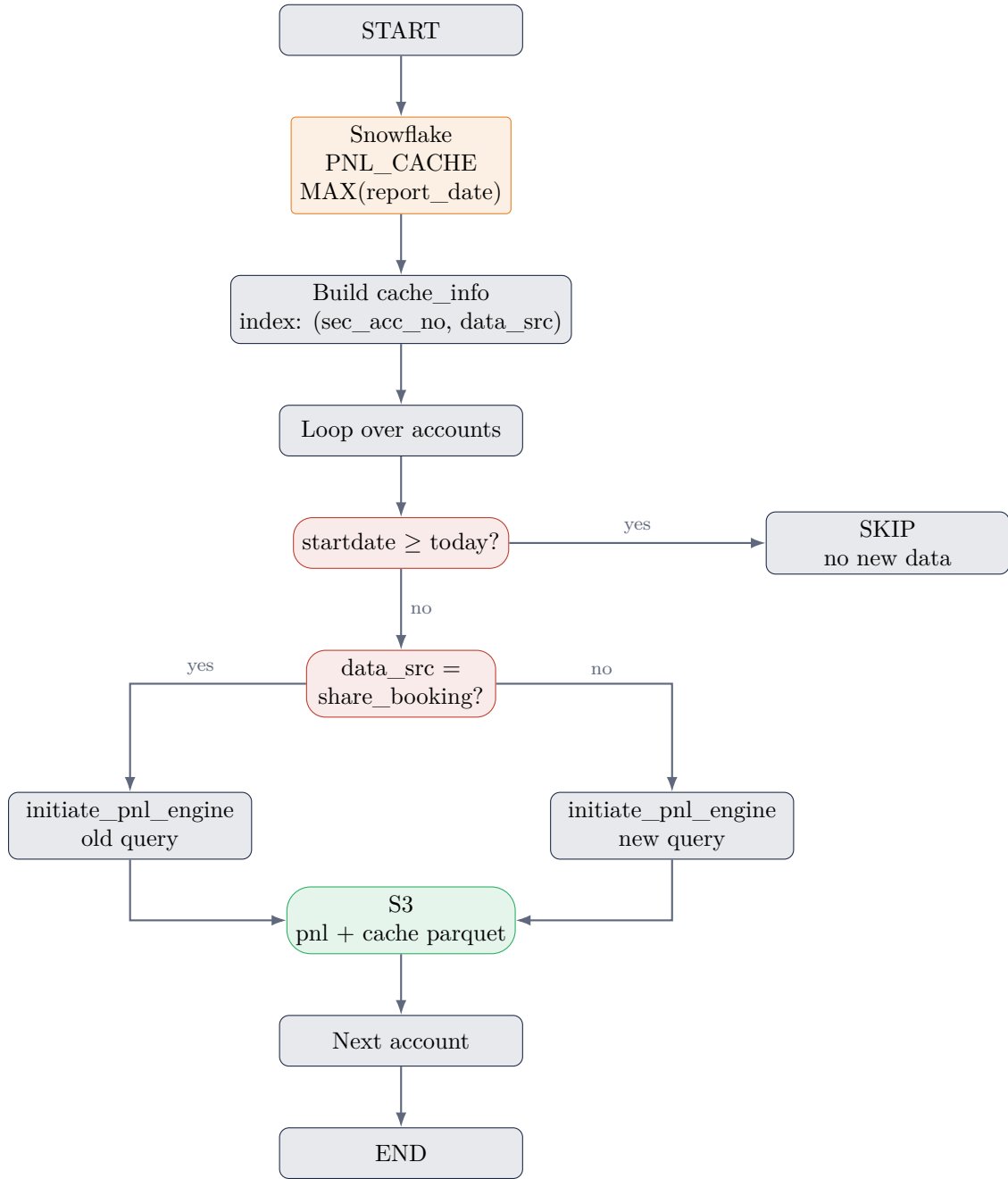


Figure 1: High-level flow of `main_run_pnl_fifo`

Incremental Date Logic

For each account the processing window is resolved as:

$$t_{\text{start}} = \underbrace{\max(\text{report_date from cache})}_{\text{last snapshot in Snowflake}} + 1 \text{ day} \quad (1)$$

$$t_{\text{end}} = \text{today} - 1 \text{ day} \quad (2)$$

If $t_{\text{start}} \geq \text{today}$ the account is skipped — no new data has arrived since the last run.

initiate_pnl_engine — Day-Loop Orchestrator

Function Signature

```
def initiate_pnl_engine(  
    acct,          # account number, e.g. 9800001301  
    tr_sd,         # trade window start date  
    tr_ed,         # trade window end date  
    ca_sd,         # cache snapshot start date  
    refetch_cache, # 1 = re-query Snowflake  
    save_data,     # 1 = write parquet at end  
    save_daily,    # 1 = write per-day parquet (historical rerun only)  
    data_src,      # 'share_booking' / 'inquisitor'  
    trade_qry,     # 'old' / 'new'  
    pth,           # output path (typically /tmp)  
    verbose        # True = print progress  
)
```

Detailed Flow

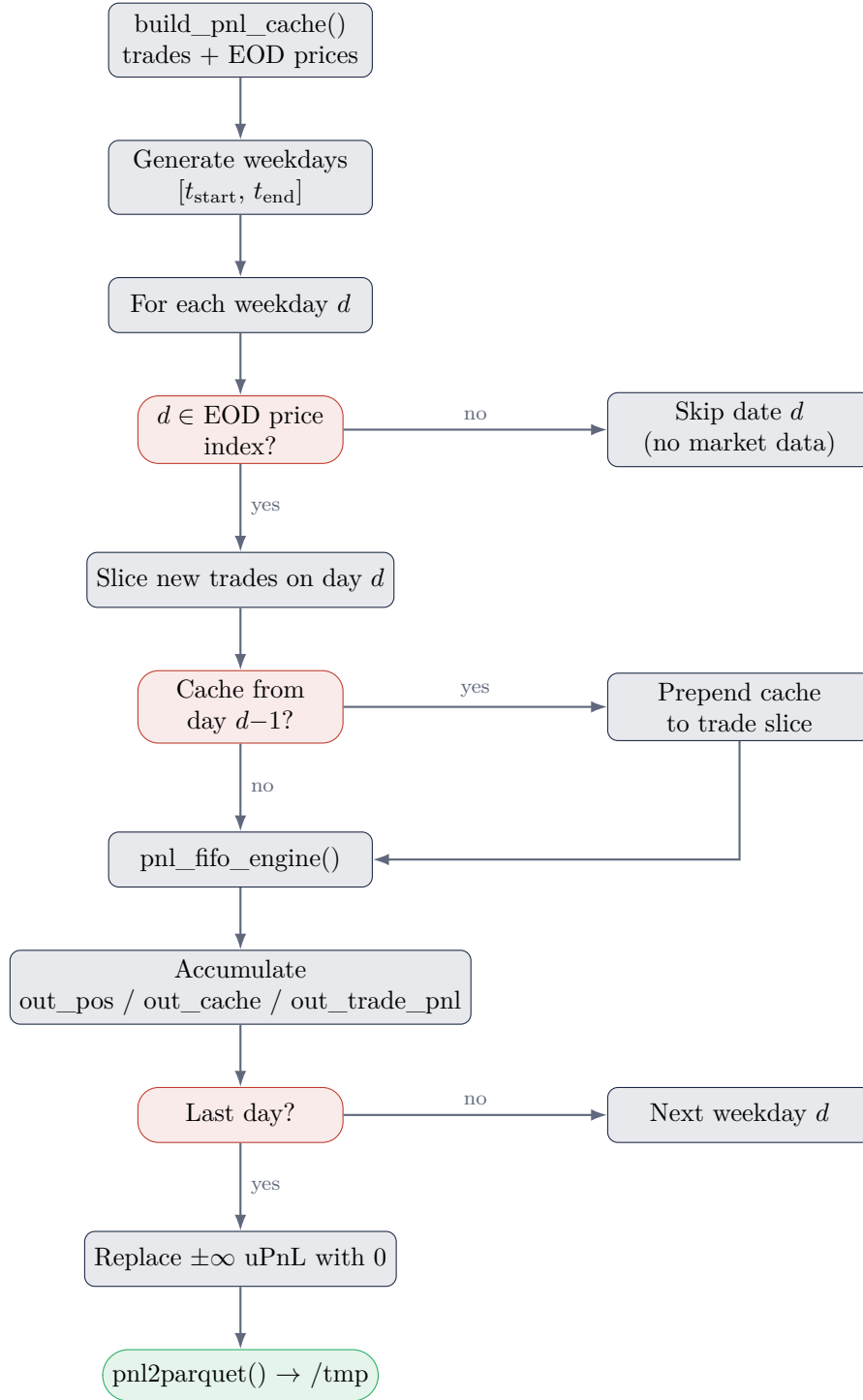


Figure 2: Day-by-day loop inside `initiate_pnl_engine`

Cache Carry-Forward

The FIFO queue must survive day boundaries. At the end of day d the `out_cache` (residual open lots) is prepended to the next day's trade slice before calling the engine:

$$\mathcal{T}^{(d)} = \text{cache}^{(d-1)} \oplus \text{new_trades}^{(d)} \quad (3)$$

where $\oplus$ denotes chronologically-sorted row concatenation. On day $d = 0$ all trades from t_{start} are

loaded at once.

Outputs

Key	Type	Content
out_pos	DataFrame	Daily positions, rPnL, uPnL per symbol
out_cache	DataFrame	Open FIFO lots to carry into next run
out_trade_pnl	DataFrame	Trade-level realised P&L
fname_pnl	str	Local path to position parquet
fname_cache	str	Local path to cache parquet

pnl_realised_polars — FIFO Calculation Engine

Role in the Call Stack

Inside `pnl_fifo_engine` the trade universe for each day is split into two groups before P&L is computed:

- **Non-CA symbols** (the majority) → `pnl_realised_polars` *(Polars-accelerated)*
- **Corporate-action symbols** → `pnl_realised_ca` *(handles split adjustments)*

Input / Output

Direction	Variable	Description
Input	df_trades	pandas DataFrame with columns: <code>symbol</code> , <code>time</code> , <code>price</code> , <code>quantity</code> , <code>side</code>
Output	out_pnl	Trade-level rPnL: <code>time</code> , <code>symbol</code> , <code>pnl</code> , <code>matched_quantity</code>
Output	out_cache	Residual open lots formatted for next-day carry-forward
Output	out_pos	Position tracker: <code>time</code> , <code>symbol</code> , <code>trade_price</code> , <code>trade_quantity</code> , <code>short_position</code>

Internal Flow

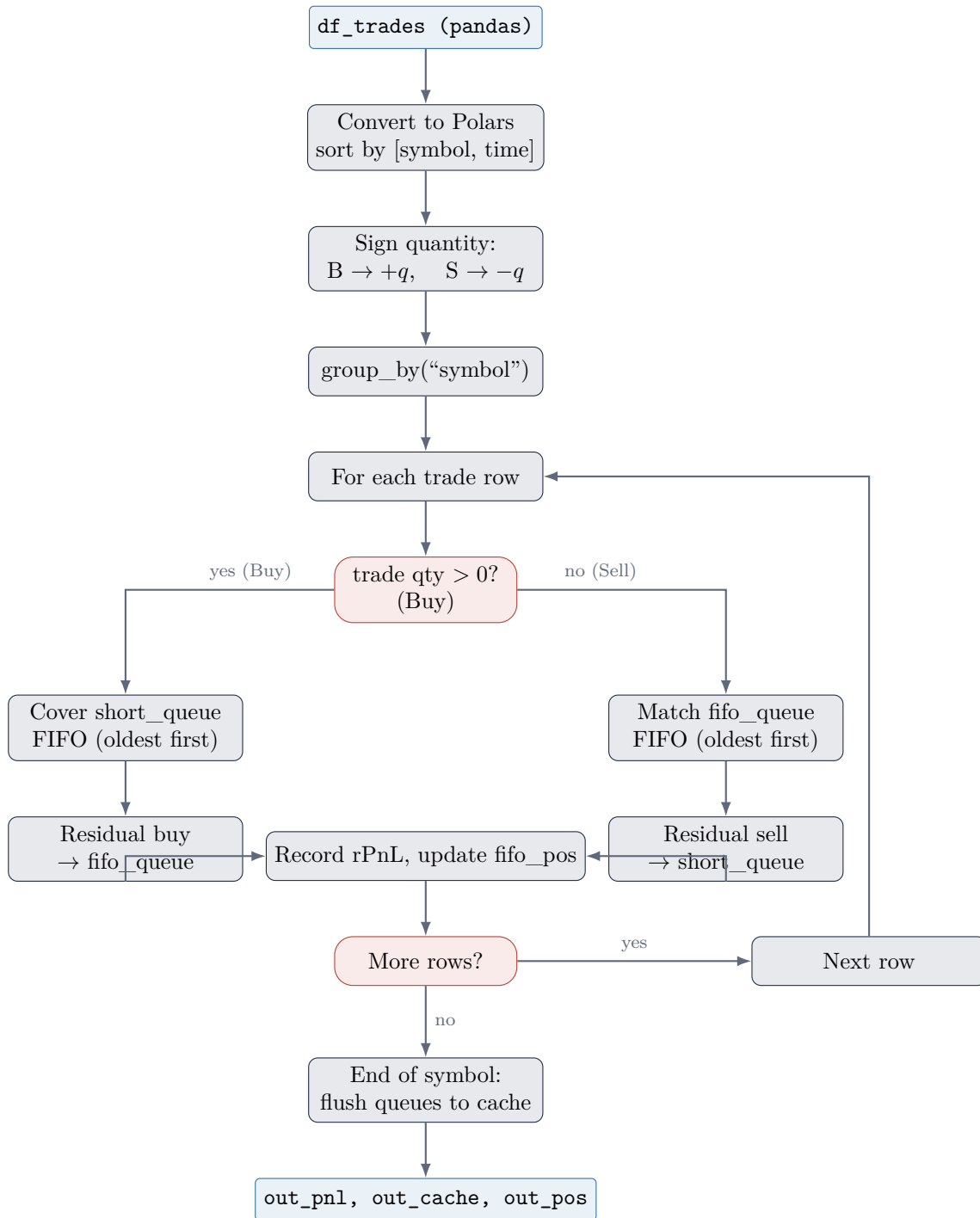


Figure 3: Internal control flow of `pnl_realised_polars`

Data Structures

Three queues are maintained *per symbol*:

Queue	Entries	Ordering	Purpose
<code>fifo_queue</code>	Unmatched buy lots	FIFO	Match against future sells
<code>short_queue</code>	Unmatched sell lots	FIFO	Match against future buys
<code>fifo_pnl</code>	P&L records	Append-only	Collect rPnL per trade

Mathematical Formulation

Sign Convention

Let trade i for symbol s have timestamp t_i , execution price p_i , and signed quantity

$$q_i = \begin{cases} +|q_i| & \text{if side} = \text{B} \quad (\text{buy}) \\ -|q_i| & \text{if side} = \text{S} \quad (\text{sell}) \end{cases} \quad (4)$$

Sell Trade ($q_i < 0$): Long Book Realisation

A sell is first matched against the **oldest** buy lots in `fifo_queue`. Let lot k carry price p_k^B and remaining quantity $q_k^B > 0$.

$$\begin{aligned} m_{ik} &= \min(|q_i^{\text{rem}}|, q_k^B) && (\text{matched quantity}) \\ \text{rPnL}_{ik} &= m_{ik} \cdot (p_i - p_k^B) && (\text{realised P\&L on sell}) \end{aligned}$$

After consuming the FIFO queue, any unmatched sell quantity is pushed to `short_queue` at price p_i :

$$q_i^{\text{short}} = |q_i| - \sum_k m_{ik} \quad (\text{if } > 0) \quad (5)$$

Buy Trade ($q_i > 0$): Short Cover

A buy first covers open short lots in `short_queue`. Let lot j carry price p_j^S and remaining quantity $q_j^S > 0$.

$$\begin{aligned} m_{ij} &= \min(q_i^{\text{rem}}, q_j^S) && (\text{matched quantity}) \\ \text{rPnL}_{ij} &= m_{ij} \cdot (p_j^S - p_i) && (\text{realised P\&L on short cover}) \end{aligned}$$

After covering shorts, any unmatched buy quantity joins the FIFO queue:

$$q_i^{\text{FIFO}} = q_i - \sum_j m_{ij} \quad (\text{if } > 0) \quad (6)$$

Total Realised P&L per Trade

$$\boxed{\text{rPnL}_i = \underbrace{\sum_j m_{ij} \cdot (p_j^S - p_i)}_{\text{short cover gain}} + \underbrace{\sum_k m_{ik} \cdot (p_i - p_k^B)}_{\text{long book gain}}} \quad (7)$$

For a pure long-only book (no shorts), only the second term is non-zero.

Unrealised P&L

After the FIFO loop the residual open lots from both queues constitute the *end-of-day open position*. Using the EOD market price P_t^{EOD} :

$$\text{uPnL}_\ell^{(t)} = (P_t^{\text{EOD}} - p_\ell) \cdot q_\ell^{\text{open}} \quad (8)$$

Aggregating over all open lots for symbol s :

$$\text{uPnL}_s^{(t)} = \sum_{\ell \in \text{open}(s)} (P_t^{\text{EOD}} - p_\ell) \cdot q_\ell^{\text{open}} \quad (9)$$

P&L Identity

Over a horizon $[t_0, T]$ the total P&L satisfies:

$$\text{Total PnL}^{(T)} = \underbrace{\sum_i \text{rPnL}_i}_{\text{cumulative realised}} + \underbrace{\text{uPnL}^{(T)}}_{\text{mark-to-market}} \quad (10)$$

This identity holds by construction: every traded unit either generated a realised P&L at the moment of matching, or remains as an open lot valued at the current market price.

FIFO Matching Examples

Standard Sell Against Long Book

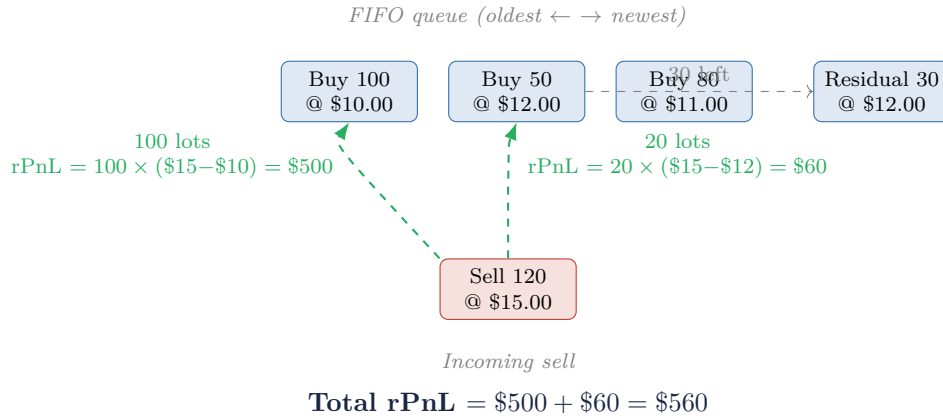


Figure 4: Sell 120 lots matched against two FIFO buy lots

Sell Exceeds Available Longs — Short Queue Created



Figure 5: Short queue created when sell quantity exceeds available FIFO lots

Key Implementation Notes

1. **Polars for I/O, Python loop for matching.** The input DataFrame is converted to Polars once and sorted via the Polars engine (fast columnar sort). The per-row FIFO matching loop itself is a plain Python `for` loop — correctness is prioritised over vectorisation for the stateful queue logic.
2. **Short-price preservation.** Each short lot in `short_queue` retains its *individual* execution price rather than being averaged. This avoids P&L mis-attribution when multiple short trades are opened at different prices and later covered individually.
3. **Corporate-action isolation.** Symbols with `booking_category == 'CORPORATE_ACTION'` are routed to `pnl_realised_ca`, which adjusts historical quantities by the split ratio before applying FIFO.
4. **Missing price handling.** Trades with a null price in the source are backfilled from the EOD price series inside `build_pnl_cache`; any still-null prices are set to 1 to prevent silent row drops.
5. **Excluded ISINs.** Three ISINs are hard-coded out of the trade feed due to known data quality issues:
US33817P3064, NL0012047823, US03815U6073.
6. **uPnL edge-case cleanup.** After the day loop, `initiate_pnl_engine` replaces $\pm\infty$ values in the uPnL column with 0; these can arise when the EOD price for an open lot is missing.

Infrastructure: AWS ECR & Airflow Orchestration

Docker Image on AWS ECR

The entire `risk_pylibrary` codebase is containerised and pushed to AWS Elastic Container Registry (ECR) on every merge to `main`:

```
263932266204.dkr.ecr.eu-central-1.amazonaws.com/risk/risk_pylibrary:main
```

The container entrypoint used by the Airflow task is:

```
python /usr/local/app/automation/task_pnl_trading_book.py
```

Resource allocation on AWS Fargate:

Resource	Request	Limit
Memory	16 GiB	16 GiB
CPU	4000 m (4 vCPU)	4000 m

The 16 GiB allocation is driven by the in-memory FIFO queue for all four accounts across a multi-month trade history loaded at once during the cache build step.

Snowflake Credentials Injection

Snowflake credentials are never baked into the image. They are injected at runtime via Kubernetes Secrets from the service account `airflow-risk-service-function-credentials`, mapped to the following environment variables inside the container:

Environment Variable	Secret Key
SNOWFLAKE_ACCOUNT	dbt-snowflake-account-id
SNOWFLAKE_USERNAME	dbt-snowflake-user
SNOWFLAKE_PASSWORD	dbt-snowflake-password
SNOWFLAKE_ROLE	dbt-snowflake-role
SNOWFLAKE_DATABASE	dbt-snowflake-database
SNOWFLAKE_WAREHOUSE	dbt-snowflake-warehouse

The pod runs under the Kubernetes service account `risk-function-models-airflow`, which carries the IAM role binding for S3 write access to `tr-risk-data-prd`.

Airflow DAG

DAG ID: `risk_function.risk_function_mrm_trading_eod_pnl`

Schedule: None (triggered externally or manually)

Owner: `Element.COMPLIANCE_AND_RISK`

Tags: `risk_function`, `risk_function_mrm`, `mrm_ecr`, `trading_book_pnl`

DAG Workflow

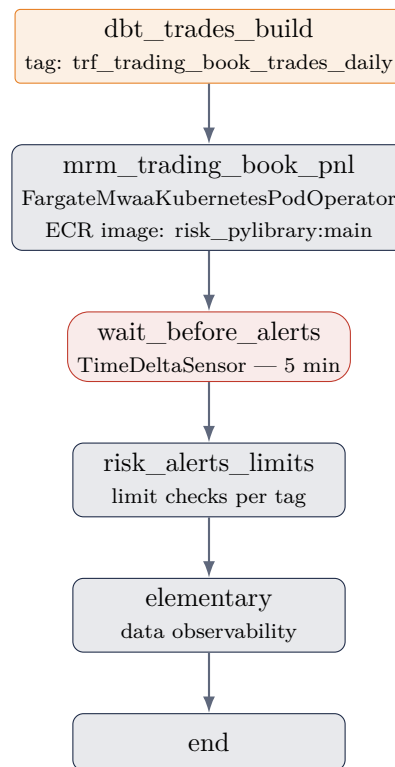


Figure 6: Airflow DAG task sequence for `risk_function_mrm_trading_eod_pnl`

Task Descriptions

1. **dbt_trades_build** — Runs `dbt build -select tag:trf_trading_book_trades_daily` via `DbtKubernetesPodOperator`. This materialises the cleaned daily trade source tables in Snowflake that `task_pnl_trading_book.py` reads.
2. **mrm_trading_book_pnl** — Pulls the ECR image and executes `task_pnl_trading_book.py` on AWS Fargate (16 GiB / 4vCPU). Startup timeout is 500s. The pod is *not* deleted on completion (`is_delete_operator_pod=False`) to allow log inspection after the run.

3. **wait_before_alerts** — A `TimeDeltaSensor` that holds the pipeline for 5 minutes after the PnL task finishes. This gives downstream Snowflake materialisation time to complete before limit checks are evaluated.
4. **risk_alerts_limits** — Runs limit-breach checks against the freshly written PnL data, filtered by tag `risk_function_mrm_trading_eod_pnl`. Generates alerts if any configured thresholds are breached.
5. **elementary** — Triggers the Elementary data-observability suite on `tags:risk_function_mrm_model_pnl` to validate data quality (row counts, freshness, schema drift).

Full Call-Stack Summary

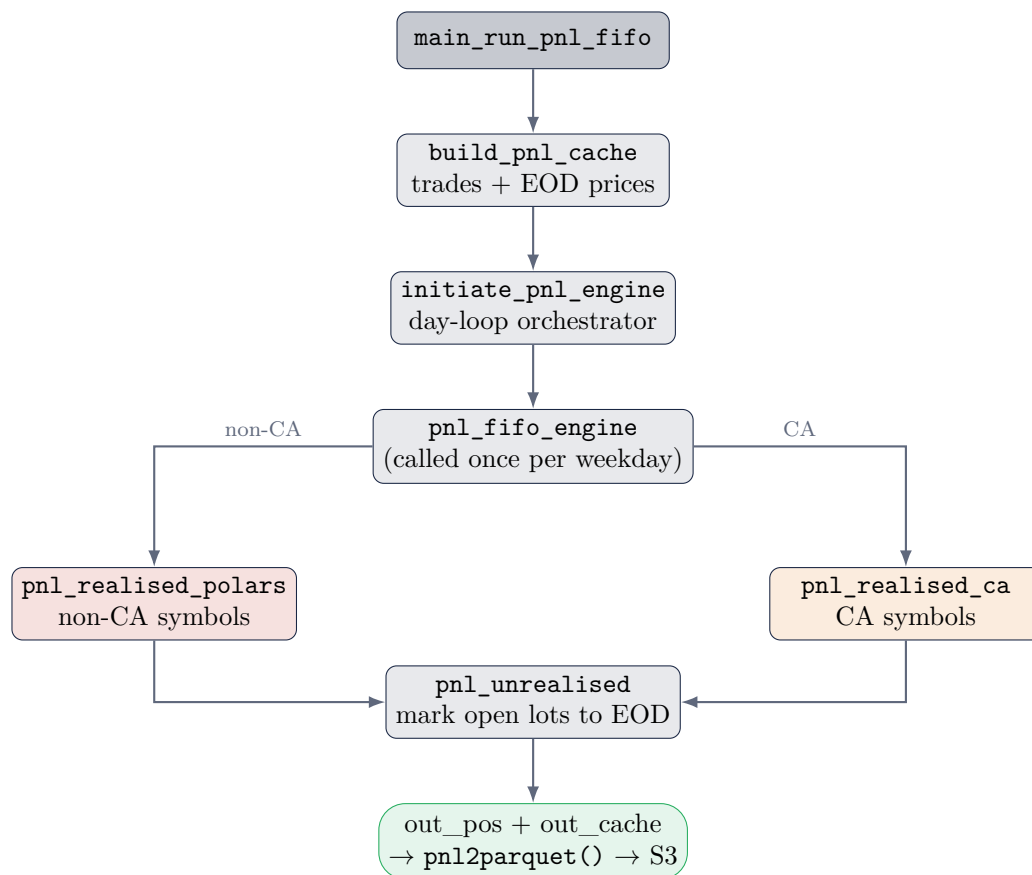


Figure 7: Complete call stack from Airflow task to S3 output

Source Code: `pnl_realised_polars`

Full source of `risk_models/pnl_fifo.py` — function `pnl_realised_polars` (lines 577–728).

```

577 def pnl_realised_polars(df_trades):
578     """
579     Calculate FIFO PnL using polars, ensuring that each residual short trade
580     retains its specific price instead of averaging.
581
582     Parameters:
583     df_trades (pd.DataFrame): Input data with columns:
584         - 'symbol': Asset identifier.
585         - 'time': Trade date.

```

```

586         - 'price': Trade execution price.
587         - 'quantity': Positive for buys, negative for sells.
588
589     Returns:
590         tuple: Three DataFrames containing:
591             - PnL results per trade
592             - Remaining FIFO queue
593             - Position tracking
594
595     """
596     # Convert to Polars DataFrame and sort by symbol and date
597     df = pl.from_pandas(df_trades).sort(by=["symbol", "time"])
598
599     # Mapping positive and negative signs for trade direction
600     df = df.with_columns(
601         pl.when(pl.col("side") == "B")
602         .then(pl.col("quantity"))
603         .otherwise(-1 * pl.col("quantity"))
604         .alias("quantity")
605     )
606
607     # Group by symbol and process each group
608     results = []
609     cache = []
610     pos = []
611
612     for group in df.group_by("symbol"):
613         trades_orig = group[1]
614         trades = trades_orig.filter(pl.col("price").is_not_null())
615
616         # Per-symbol queues
617         fifo_queue = [] # unmatched buy lots {time, symbol, quantity, price}
618         fifo_pnl = [] # realised PnL records
619         fifo_pos = [] # position tracker
620         short_queue = [] # individual short lots (price preserved, not
621                         # averaged)
622
623         for row in trades.iter_rows(named=True):
624             trade_quantity = row["quantity"]
625             trade_price = row["price"]
626             realized_pnl = 0.0
627             match_quantity = 0.0
628
629             # -----
630             # BUY trade: cover shorts first, then enqueue remainder
631             # -----
632             if trade_quantity > 0:
633                 while trade_quantity > 0 and short_queue:
634                     short_trade = short_queue[0]
635                     match_quantity = min(trade_quantity, short_trade["quantity"])
636
637                     # Short cover: gain = (short price - buy price) * matched qty
638                     realized_pnl += match_quantity * (short_trade["price"] - trade_price)
639
640                     short_trade["quantity"] -= match_quantity
641                     trade_quantity -= match_quantity
642
643                     if short_trade["quantity"] == 0:
644                         short_queue.pop(0)
645
646             fifo_pos.append({
647                 "time": row["time"],
648                 "symbol": row["symbol"],
649                 "trade_price": trade_price,

```

```

647         "trade_quantity": trade_quantity,
648         "short_position": sum(t["quantity"] for t in short_queue),
649         "match_quantity": match_quantity,
650     })
651
652     # Remaining buy quantity -> FIFO queue
653     if trade_quantity > 0:
654         fifo_queue.append({
655             "time": row["time"],
656             "symbol": row["symbol"],
657             "quantity": trade_quantity,
658             "price": trade_price,
659         })
660
661     # -----
662     # SELL trade: match against FIFO queue, overflow -> short queue
663     # -----
664     elif trade_quantity < 0:
665         sell_quantity = -trade_quantity
666
667         while sell_quantity > 0 and fifo_queue:
668             buy_trade = fifo_queue[0]
669             match_quantity = min(sell_quantity, buy_trade["quantity"])
670             # Long realisation: gain = (sell price - buy price) *
671             #     matched qty
672             realized_pnl += match_quantity * (trade_price - buy_trade[
673                 "price"])
674
675             buy_trade["quantity"] -= match_quantity
676             sell_quantity -= match_quantity
677
678             if buy_trade["quantity"] == 0:
679                 fifo_queue.pop(0)
680
681         # Unmatched sell residual stored with its own price (not
682         #     averaged)
683         if sell_quantity > 0:
684             short_queue.append({
685                 "time": row["time"],
686                 "symbol": row["symbol"],
687                 "quantity": sell_quantity,
688                 "price": trade_price,
689             })
690
691         fifo_pos.append({
692             "time": row["time"],
693             "symbol": row["symbol"],
694             "trade_price": trade_price,
695             "trade_quantity": trade_quantity,
696             "short_position": sum(t["quantity"] for t in short_queue),
697             "match_quantity": match_quantity,
698         })
699
700     # Record PnL for this trade (0.0 if no match occurred)
701     fifo_pnl.append({
702         "time": row["time"],
703         "symbol": row["symbol"],
704         "pnl": realized_pnl,
705         "matched_quantity": match_quantity,
706     })
707
708     # Flush short_queue into cache with negated quantities (sign convention
709     #     )
710     if len(short_queue) > 0:
711         short_queue = [{"symbol": s, "quantity": -1 * p["quantity"]} for s, p in

```

```
        short_queue]
708
709     cache.extend(short_queue)
710     results.extend(fifo_pnl)
711     cache.extend(fifo_queue)
712     pos.extend(fifo_pos)
713
714     # Convert results back to pandas
715     out_pnl = pd.DataFrame(results)
716     out_cache = pd.DataFrame(cache)
717     out_pos = pd.DataFrame(pos)
718
719     # Format cache for next-day carry-forward
720     out_cache["side"] = np.where(out_cache["quantity"] < 0, "S", "B
        ")
721     out_cache["booking_type"] = np.where(out_cache["quantity"] < 0, "SELL",
        "BUY")
722     out_cache["quantity"] = out_cache["quantity"].abs()
723     out_cache["booking_category"] = "TRADING"
724     out_cache["multiplier"] = 1
725     out_cache = out_cache[[
726         "time", "symbol", "side", "booking_category",
727         "booking_type", "price", "quantity", "multiplier",
728     ]]
729
730     return out_pnl, out_cache, out_pos
```